

OOP

UNIT 10 – GENERIC PROGRAMMING WITH TEMPLATES

When the algorithm is the same but the type changes, the type should become a parameter.

FROM REPEATED CODE TO GENERIC DESIGN

The previous units showed several ways in which C++ lets one design work with more than one concrete type. Function and operator overloading allow the compiler to select among several functions. Inheritance and virtual functions allow code to use a common base interface while the real object chooses behavior at run time.

This unit addresses a different situation. Sometimes there are not several different behaviors at all. The algorithm is exactly the same, but we keep rewriting it only because the data type changes. Generic programming lets us write that algorithm once and make the type itself a parameter.

THE REPEATED MAXIMUM PROBLEM

Suppose your application needs a small utility that returns the larger of two values. The first version works with int:

```
int larger(int a, int b)
{
    return (a > b) ? a : b;
}
```

Later we need the same operation for double, so we add another overload:

```
double larger(double a, double b)
{
    return (a > b) ? a : b;
}
```

Nothing about the algorithm changed. Only the type changed. If the same need appears for long, float, or a user-defined class, ordinary overloading can keep adding versions, but the body of the function is duplicated again and again.

The design question for this unit is therefore simple: if one algorithm is correct for many types, why should the source code contain many copies of that algorithm?

LEARNING OBJECTIVES

By the end of this unit, you should be able to:

- Explain the problem that generic programming and templates solve.
- Write and use a function template with one or more type parameters.
- Explain template argument deduction and explicit template arguments.
- Identify the operations that a template requires from its type parameter.
- Explain how operator overloading allows user-defined classes to participate in generic algorithms.
- Define, instantiate, and use a class template.
- Explain why template definitions are normally visible in header files.
- Distinguish compile-time polymorphism from run-time polymorphism and explain when each design is appropriate.
- Recognize class templates such as `vector<T>` as the foundation of reusable STL containers.

OUR FIRST FUNCTION TEMPLATE

A function template describes a family of functions. Instead of writing one function for `int` and another for `double`, we write one pattern and use a type parameter wherever the concrete type would normally appear.

```
template <typename T>
T larger(T a, T b)
{
    return (a > b) ? a : b;
}
```

The identifier `T` is not a special built-in type. It is a placeholder chosen by the programmer. When the template is used, the compiler determines which concrete type should replace `T` and creates the corresponding function.

READING TEMPLATE <typename T>

The first line introduces the template parameter:

```
template <typename T>
```

Read it as: the following declaration is a template, and T represents a type that will be supplied later. Inside the function, every occurrence of T refers to that same type.

- The return type T means the function returns a value of the chosen type.
- The parameters T a and T b mean both arguments are expected to use that type.
- The body is written exactly once, independently of whether T later becomes int, double, string, or a suitable class type.

C++ also permits the keyword class in this position:

```
template <class T>
```

For a simple type template parameter, class and typename mean the same thing. This course will use typename because it states the intent more clearly: T stands for a type.

USING THE FUNCTION TEMPLATE

```
cout << larger(3, 7) << endl;  
cout << larger(2.5, 4.1) << endl;
```

The calls look like ordinary function calls. The compiler examines the arguments and deduces the template type automatically.

```
7  
4.1
```

Conceptually, the first call causes the compiler to instantiate a version in which *T* is *int*. The second call instantiates another version in which *T* is *double*. The source code contains one function template, while the compiler creates the concrete functions that are actually needed.

TEMPLATE INSTANTIATION

Instantiation is the process of producing a *concrete* function or class from a template. The compiler does not treat T as a run-time variable. T is resolved during compilation.

```
larger(3, 7);           // T becomes int  
larger(2.5, 4.1);      // T becomes double
```

A useful mental model is:

```
template larger<T>  
    |  
    +--> larger<int>  
    |  
    +--> larger<double>
```

This is one reason templates are described as a form of compile-time polymorphism: the compiler creates and selects the required version before the program runs.

TEMPLATE ARGUMENT DEDUCTION

Usually we let the compiler infer the template argument from the function arguments. This is called template argument *deduction*.

```
int x = larger(10, 20);           // T = int
double y = larger(1.5, 2.0);     // T = double
```

Deduction succeeds because both arguments agree on the same T. A call with conflicting argument types is different:

```
larger(3, 4.5);    // Error: should T be int or double?
```

For this one-parameter template, the first argument suggests int and the second suggests double. The compiler will not silently invent a common T for this template declaration.

EXPLICIT TEMPLATE ARGUMENTS

When necessary, the programmer can state the template argument *explicitly* between angle brackets:

```
cout << larger<double>(3, 4.5) << endl;
```

Now T is fixed as double before the arguments are checked. The integer 3 can be converted to double, so the call is valid. Explicit template arguments are useful in special cases, but ordinary deduction usually produces clearer code.

PAUSE AND PREDICT

Assume the larger template from the previous section. Decide whether each line compiles and, if it does, identify T:

```
larger(4, 9);  
larger(2.0, 7.5);  
larger('a', 'z');  
larger(4, 7.5);  
larger<double>(4, 7.5);
```

The first three calls deduce int, double, and char. The fourth cannot deduce one T from two different argument types. The final call fixes T as double and therefore succeeds after converting 4 to double.

A TEMPLATE IS NOT AUTOMATICALLY VALID FOR EVERY TYPE

The syntax template <typename T> does not promise that every possible type will work. The function body still uses real operations, and the selected type must support those operations.

```
template <typename T>
T larger(T a, T b)
{
    return (a > b) ? a : b;
}
```

This template requires an expression of the form $a > b$. Therefore T must support a meaningful operator>. Built-in numeric types do. A user-defined class works only if its public interface provides the required comparison.

TEMPLATES AND OPERATOR OVERLOADING

Operators' second purpose: they let the class participate in generic code.

```
class Song {
private:
    string title;
    int duration;
public:
    Song(const string& title, int duration)
        : title(title), duration(duration) { }
    //some more code
    bool operator>(const Song& other) const {
        return duration > other.duration;
    }
};
```

Because Song now supports operator>, larger template can compare two songs:

```
Song shortSong("Yesterday", 123);
Song longSong("Imagine", 183);
Song result = larger(shortSong, longSong);
```

GENERIC PROGRAMMING IS BASED ON REQUIREMENTS

This leads to a central rule of generic programming: a template should be understood in terms of the operations it requires, not merely in terms of the letter T.

For `larger<T>`, the important requirement is not "T must be numeric." The real requirement is: two T values can be compared with `>`, and a T value can be returned from the function. That requirement is satisfied by numeric types and by many user-defined types.

Thinking in terms of required operations makes templates more reusable and also explains many template compiler errors. If one required operation is missing, the error often appears when the template is instantiated for that particular type.

A MORE USEFUL EXAMPLE: CONTAINS

Your application needs to ask whether a value already appears in a collection. We could write one function for `int`, another for `string`, and another for `Song`. The loop would be *identical* each time.

```
template <typename T>
bool contains(const T values[], int size, const T& target) {
    for (int i = 0; i < size; ++i)    {
        if (values[i] == target)
            return true;
    }
    return false;
}
```

This template requires operator`==` for `T`. It does not care whether equality means numeric equality, equal strings, or a class-specific definition such as equal song titles.

USING CONTAINS WITH DIFFERENT TYPES

```
int numbers[] = { 10, 20, 30, 40 };
string names[] = { "Ana", "Ben", "Maya" };

cout << boolalpha;
cout << contains(numbers, 4, 30) << endl;
cout << contains(names, 3, string("Ben")) << endl;

true
true
```

One source definition has now produced useful behavior for two unrelated types. The same idea extends naturally to classes when their public operations satisfy the template requirements.

WHY CONST T& APPEARS SO OFTEN

The target parameter in contains is written as const T&:

```
const T& target
```

This combines two ideas learned earlier. Passing by reference avoids an unnecessary copy when T is a large object. const guarantees that contains cannot modify the target while searching.

Generic code should not assume that T is a tiny built-in type. T might be int, but it might also be a Song, an image object, or another class with expensive state.

Using const references where appropriate helps the same template remain efficient for both small and large types.

FUNCTION TEMPLATES WITH MORE THAN ONE TYPE

A template may have several type parameters when different parts of the algorithm are allowed to use different types.

```
template <typename T, typename U>
void printLabeledValue(const T& label, const U& value) {
    cout << label << ": " << value << endl;
}

printLabeledValue(string("Plays"), 1250);
printLabeledValue(string("Rating"), 4.8);
```

The first call can deduce T as string and U as int. The second can deduce T as string and U as double. The template still relies on a *requirement*: both values must be printable with operator<< in the expressions used by the function.

FUNCTION TEMPLATES AND ORDINARY OVERLOADS CAN COEXIST

A program is not forced to choose between templates and ordinary functions. A special case may deserve a dedicated overload while the general case remains templated.

```
template <typename T>
void printValue(const T& value)
{
    cout << value << endl;
}

void printValue(const Song& song)
{
    cout << "Song duration: " << song.getDuration() << endl;
}
```

The ordinary Song overload expresses special behavior for Song. The template provides the general behavior for other printable types. The compiler applies its overload-resolution rules to choose the best match.

FROM FUNCTION TEMPLATES TO CLASS TEMPLATES

Function templates make an *algorithm* independent of one concrete type.

Class templates apply the same idea to an entire class design. Instead of writing separate container classes for integers, strings, and songs, we can parameterize the stored type.

Imagine maintaining these three classes:

```
class IntPair;  
class StringPair;  
class SongPair;
```

If every class stores two values and provides the same operations, only the member type changes. This is exactly the kind of duplication a class template is designed to remove.

OUR FIRST CLASS TEMPLATE

```
template <typename T>
class Pair {
private:
    T first;
    T second;
public:
    Pair(const T& first, const T& second)
        : first(first), second(second) { }
    const T& getFirst() const {
        return first;
    }
    const T& getSecond() const {
        return second;
    }
};
```

The class definition is a *pattern*. Pair by itself is not yet one concrete class. A concrete type is formed when a template argument is supplied.

INSTANTIATING A CLASS TEMPLATE

```
Pair<int> scores(85, 92);  
Pair<string> artists("Adele", "Sting");  
Pair<Song> songs(shortSong, longSong);
```

`Pair<int>`, `Pair<string>`, and `Pair<Song>` are three different concrete types generated from the same class template. The compiler substitutes the chosen type for every occurrence of `T` in the class definition.

The angle brackets are part of the type name. Once `Pair` is a template, writing only `Pair` where a complete type is required is normally incomplete because the compiler still needs to know which `T` the class should store.

A CLASS TEMPLATE WITH TWO TYPE PARAMETERS

Class templates may also use different types for different members. A labeled value in the music application could pair a string-like key with a value of another type.

```
template <typename Key, typename Value>
class Entry {
private:
    Key key;
    Value value;
public:
    Entry(const Key& key, const Value& value)
        : key(key), value(value) { }

    const Key& getKey() const { return key; }
    const Value& getValue() const { return value; }
};

Entry<string, int> playCount("Imagine", 12547);
Entry<string, double> rating("Imagine", 4.9);
```

The template parameter names Key and Value are more descriptive than T and U. Short names are common in small templates, while meaningful names can make larger generic classes easier to read.

DEFINING CLASS-TEMPLATE MEMBERS OUTSIDE THE CLASS

Member functions may be defined inside the class body, as in the Pair example. They may also be written after the class declaration. The syntax is more detailed because the compiler must be reminded that both the class and the function definition depend on T.

```
template <typename T>
class Pair
{
private:
    T first;
    T second;

public:
    Pair(const T& first, const T& second);
    const T& getFirst() const;
```

```
};
```

The constructor definition becomes:

```
template <typename T>
Pair<T>::Pair(const T& first, const T& second)
    : first(first), second(second)
{
}
```

The accessor definition follows the same pattern:

```
template <typename T>
const T& Pair<T>::getFirst() const
{
    return first;
}
```

READING PAIR<T>::

The scope operator now uses `Pair<T>::` rather than `Pair::`. This is necessary because `Pair` is a family of classes. `Pair<T>::` means the member that belongs to the `Pair` class instantiated with the current template type `T`.

- `template <typename T>` introduces `T` for this definition.
- `Pair<T>::` identifies the current `Pair<T>` class whose member is being defined.
- The remaining function syntax is ordinary C++ member-function syntax.

Forgetting either the template line or the `<T>` after the class name is one of the most common beginner errors in class-template definitions.

A GENERIC COLLECTION

A pair demonstrates class-template syntax, but a collection shows why class templates are so useful in larger programs. Earlier units implemented classes that own dynamic arrays and follow the Rule of Three. We can now parameterize the element type and reuse that design.

```
template <typename T>
class Collection {
private:
    T* items;
    int size;
    int capacity;
public:
    explicit Collection(int capacity = 10);
    Collection(const Collection<T>& other);
    Collection<T>& operator=(const Collection<T>& other);
    ~Collection();

    bool add(const T& item);
```

```
int getSize() const;
T& operator[](int index);
const T& operator[](int index) const;
};
```

Only one word in the design is truly unknown: the type of item stored by the collection. Everything else—capacity, copying, indexing, destruction, and the add operation—follows the same object-oriented rules learned earlier.

CONSTRUCTING AND DESTROYING COLLECTION<T>

```
template <typename T>
Collection<T>::Collection(int capacity)
    : items(new T[capacity]), size(0), capacity(capacity)
{ }

template <typename T>
Collection<T>::~~Collection() {
    delete[] items;
}
```

This is familiar resource management expressed generically. If T is int, the dynamic array stores integers. If T is Song, it stores Song objects. The same constructor and destructor source code is reused.

This particular teaching implementation uses new T[capacity]. That expression requires T to be default-constructible. Generic code often has such requirements even when they are not written explicitly in the template parameter list.

COPY CONSTRUCTION FOR COLLECTION<T>

```
template <typename T>
Collection<T>::Collection(const Collection<T>& other)
    : items(new T[other.capacity]),
      size(other.size),
      capacity(other.capacity)    {
    for (int i = 0; i < size; ++i)
        items[i] = other.items[i];
}
```

The loop does not know whether it is copying integers, strings, or songs. It simply requires that T can be assigned.

ASSIGNMENT FOR COLLECTION<T>

```
template <typename T>
Collection<T>& Collection<T>::operator=(const Collection<T>&
other) {
    if (this == &other)
        return *this;

    T* newItems = new T[other.capacity];

    for (int i = 0; i < other.size; ++i)
        newItems[i] = other.items[i];

    delete[] items;

    items = newItems;
    size = other.size;
    capacity = other.capacity;

    return *this;
}
```

Nothing about the Rule of Three disappeared when the class became a template. Generic programming changes which types the class can work with; it does not remove the class's responsibility for correct ownership and copying.

ADDING AND READING ELEMENTS

```
template <typename T>
bool Collection<T>::add(const T& item) {
    if (size == capacity)
        return false;

    items[size++] = item;
    return true;
}

template <typename T>
int Collection<T>::getSize() const {
    return size;
}
```

Passing item as `const T&` avoids an unnecessary copy at the call boundary. The actual copy occurs only when the collection stores the new element in its own array.

A GENERIC SUBSCRIPT OPERATOR

Because operator overloading was already introduced earlier, the generic collection can support familiar indexing syntax for any stored type.

```
template <typename T>
T& Collection<T>::operator[](int index)
{
    return items[index];
}

template <typename T>
const T& Collection<T>::operator[](int index) const
{
    return items[index];
}
```

The non-const overload permits modification when the collection itself is non-const. The const overload permits reading from a const collection without allowing the selected element to be modified through that reference.

USING THE SAME COLLECTION WITH DIFFERENT TYPES

```
Collection<int> numbers(5);  
numbers.add(10);  
numbers.add(20);  
  
Collection<string> names(5);  
names.add("Ana");  
names.add("Ben");  
  
Collection<Song> songs(5);  
songs.add(shortSong);  
songs.add(longSong);
```

The public interface is identical in all three cases. The compiler creates different Collection instantiations because the stored element types are different. Collection<int> and Collection<Song> are distinct types even though they come from the same template.

WHAT DOES COLLECTION<T> REQUIRE FROM T?

The implementation itself reveals its requirements:

- `new T[capacity]` requires T to be default-constructible in this simplified design.
- `items[i] = other.items[i]` requires assignment from another T.
- `add` stores an item by assignment, so T must again support assignment.
- The collection destructor invokes the destructor of every array element automatically, so T must be safely destructible.

These requirements are not arbitrary restrictions imposed by templates. They follow directly from the operations written in the class. If we change the implementation, the requirements may change as well.

TEMPLATE DEFINITIONS AND HEADER FILES

Ordinary classes in this course were usually divided into a header containing declarations and a .cpp file containing implementations. Templates require special attention because the compiler normally needs to see the complete template definition when it instantiates a new type.

For that reason, a common organization is to place both the declaration and the template member definitions in the header file:

```
// Collection.h
#ifndef COLLECTION_H
#define COLLECTION_H

template <typename T>
class Collection
{
    // declaration
};

template <typename T>
```

```
Collection<T>::Collection(int capacity)
{
    // implementation
}

#endif
```

Another common project style places the definitions in a separate implementation file such as `Collection.cpp` and includes that file at the end of the header. The important principle is visibility: the compiler must have the template definition available when a concrete `Collection<T>` is instantiated.

WHY THE USUAL .H/.CPP SEPARATION CAN FAIL

Suppose `Collection.h` contains only declarations and `Collection.cpp` contains the template definitions. Another source file includes `Collection.h` and writes `Collection<Song>`. The compiler compiling that source file sees that a `Collection<Song>` is needed, but it may not see the definitions from which that instantiation should be generated.

This often leads to confusing linker errors even though the declarations look correct. The safest beginner rule is: keep function-template and class-template definitions visible in the header unless you are deliberately using a more advanced explicit-instantiation design.

COMPILE-TIME POLYMORPHISM

Templates are often described as *compile*-time polymorphism. The same source-level operation can work with different types, but the type is determined during compilation.

In this course, the main compile-time mechanisms include:

- Function overloading: the compiler chooses among functions with the same name but different parameter lists.
- Operator overloading: the compiler selects the operator function appropriate for the operand types.
- Templates: the compiler instantiates and selects a function or class instantiation for the supplied type arguments.

No vtable lookup is required for the template decision itself. The concrete type is known before the program begins executing.

RUN-TIME POLYMORPHISM

Run-time polymorphism solves a different problem. A base reference or pointer may refer to objects of several derived types, and a virtual call selects the override according to the real object at run time.

```
MediaItem* item = &song;  
item->play();
```

The declared pointer type is `MediaItem*`, but the real object may be `Song`, `PodcastEpisode`, `Audiobook`, or another concrete derived type. The `vptr/vtable` mechanism discussed earlier determines which override runs.

TEMPLATES VS. VIRTUAL FUNCTIONS

The two mechanisms should not be treated as competitors. They answer different design questions.

Use a template when the algorithm or class design is essentially the same and only a type changes. The compiler can generate a type-specific version from one generic pattern.

Use inheritance and virtual functions when several concrete object types share a common interface but must provide different behavior that is selected according to the real object at run time.

A useful test is to ask: "Do I want the same implementation for many types, or different implementations behind one common interface?" The first question points toward templates. The second points toward run-time polymorphism.

TEMPLATES AND RUN-TIME POLYMORPHISM CAN WORK TOGETHER

A generic container can store base-class pointers and then use virtual dispatch for the pointed-to objects. This combines compile-time and run-time polymorphism in one design.

```
Collection<MediaItem*> library(10);  
  
library.add(&song);  
library.add(&episode);  
library.add(&book);  
  
for (int i = 0; i < library.getSize(); ++i)  
    library[i]->play();
```

`Collection<MediaItem*>` is chosen and instantiated at compile time. During the loop, each play call is virtual, so the real pointed-to object selects `Song::play`, `PodcastEpisode::play`, or `Audiobook::play` at run time.

As in the earlier `MusicLibrary` design, these pointers are non-owning in this example. The objects must remain alive while the collection refers to them.

ONE TEMPLATE INSTANTIATION IS ONE CONCRETE TYPE

It is important not to confuse a class template with an inheritance hierarchy. `Collection<int>` and `Collection<string>` are different types. The fact that both were produced from `Collection<T>` does not create an is-a relationship between them.

```
Collection<int> numbers;  
Collection<string> names;  
  
// numbers = names;    // Error: different types
```

Templates provide a common pattern for generating types; inheritance provides relationships among types. These are separate concepts even though both support reusable designs.

A MISSING OPERATION PRODUCES AN INSTANTIATION ERROR

Consider this template:

```
template <typename T>
bool same(const T& a, const T& b)
{
    return a == b;
}
```

The template declaration itself can be accepted before the compiler knows which T will be used. If a later call instantiates same for a class that has no usable operator==, compilation fails for that instantiation.

This explains why template error messages sometimes appear far from the original call. The compiler is tracing an operation required by the instantiated template. When reading such an error, ask which expression in the template cannot be formed for the chosen type.

GUIDED PRACTICE: COUNTMATCHES

Write a generic function that counts how many elements of an array are equal to a target. The function should avoid copying the target and should work for any type that supports operator==.

```
template <typename T>
int countMatches(const T values[], int size, const T& val)
{
    int count = 0;

    for (int i = 0; i < size; ++i)
    {
        if (values[i] == val)
            ++count;
    }

    return count;
}
```

Now test the same source definition with integers and strings. Then ask what must be added to Song before the same template can count matching songs. The answer is not a special template feature; Song simply needs a meaningful `operator==` that satisfies the algorithm's requirement.

INDEPENDENT CHECK: DESIGN BEFORE CODING

A statistics application needs a class that stores a minimum value and a maximum value of the same type. The design should work for `int`, `double`, and any comparable user-defined type.

Without writing a full program, answer these questions:

1. What should the template parameter represent?
2. Which two data members should the class store?
3. What constructor parameters should be passed by const reference?
4. If the class includes a `contains(value)` function using `>=` and `<=`, which operators must `T` support?
5. Would `Range<int>` and `Range<double>` be the same concrete type or different types?

DESIGN GUIDELINES

- Use a template when the algorithm or class structure is the same across types.
- Describe a template by the operations it requires from its type parameters.
- Prefer const references for read-only parameters that may represent large objects.
- Do not assume that every type supports operators such as `==`, `<`, `>`, or `<<`; require only what the implementation really uses.
- Remember that `Class<int>` and `Class<double>` are different concrete types.
- Keep template definitions visible where instantiation occurs, normally in a header..
- Use templates for same-design/different-type reuse and virtual functions for different-behavior/one-interface run-time selection.
- Keep ownership rules explicit when a generic container stores pointers.
- Prefer standard library containers such as `vector<T>` in production code when they fit the problem; the teaching `Collection<T>` exists to expose the underlying ideas.

COMMON BEGINNER MISTAKES

- Thinking `T` is a special built-in C++ type .
- Assuming a template automatically works for every possible type.
- Calling a one-type-parameter template with arguments that deduce conflicting types.
- Writing `Pair` instead of `Pair<T>` when defining a member outside a class template.
- Forgetting template `<typename T>` before an out-of-class class-template member definition.
- Moving template definitions into an ordinary `.cpp` file and then encountering unresolved linker errors.
- Confusing class-template instantiations with derived classes.
- Using pass-by-value everywhere and accidentally copying large objects.
- Assuming templates replace inheritance or virtual functions; the two mechanisms solve different design problems.

SUMMARY

Generic programming removes duplication that exists only because a type changes. A function template defines a family of functions, and a class template defines a family of classes. In both cases, the template parameter represents a type that is supplied or deduced during compilation.

Template instantiation produces concrete code such as `larger<int>`, `Pair<string>`, or `Collection<Song>`. The template is reusable only for types that support the operations its implementation actually uses. Operators, constructors, assignment, and stream insertion are therefore part of the practical requirements of generic code.

Class templates do not replace the object-oriented ideas learned earlier. Constructors, destructors, the Rule of Three, `const` correctness, references, encapsulation, and operator overloading remain fully relevant inside a class template. Generic programming parameterizes the element type while preserving those design responsibilities.

Templates provide compile-time polymorphism; virtual functions provide run-time polymorphism. The two can be combined—for example, a

Collection<MediaItem*> can be generated at compile time and still dispatch play() virtually at run time. Understanding the distinction prepares us to recognize the design of the Standard Template Library.

REVIEW QUESTIONS

1. What problem does generic programming solve when several functions contain the same algorithm but use different types?
2. What does template `<typename T>` declare?
3. What is template instantiation?
4. How does template argument deduction determine `T` in `larger(3, 7)`?
5. Why does `larger(3, 4.5)` fail for a template that expects both parameters to have the same `T`?
6. How can an explicit template argument make `larger<double>(3, 4.5)` valid?
7. Why does `larger<T>` require `operator>` from `T`?
8. How can operator overloading make `Song` usable with a generic comparison function?
9. Why is `const T&` commonly used in function templates?
10. What is the difference between a function template and a class template?
11. Are `Pair<int>` and `Pair<string>` the same type? Explain.

12. Why must an out-of-class member definition use both template <typename T> and Pair<T>::?
13. Which earlier object-oriented concepts still apply inside Collection<T>?
14. Why are template definitions normally kept visible in a header file?
15. How does compile-time polymorphism differ from run-time polymorphism?
16. How can Collection<MediaItem*> combine templates with virtual dispatch?

EXIT CHECK

Complete these four sentences without looking back:

1. A function template allows me to write ... once while making ... a parameter.
2. A class template becomes a concrete type when
3. A template works for a type only if that type supports
4. Templates select type-specific code at ..., while virtual functions select an override at

LOOKING AHEAD

We can now write algorithms and classes that are independent of one specific data type. That idea is not an isolated C++ feature; it is the foundation of one of the most important parts of modern C++ programming.

The Standard Template Library provides reusable class templates and function templates that have already solved many of the problems we have been implementing manually. A `vector<int>`, `vector<string>`, and `vector<Song>` all come from the same generic container design. Iterators provide a common way to move through containers, and generic algorithms operate on ranges without knowing the concrete element type in advance.

In the next unit, we will begin working with the STL. The goal will not be to memorize a large library, but to recognize how the generic-programming ideas from this unit appear in professionally designed containers, iterators, and algorithms.